

Testes

PROF. LÁZARO

Desenvolvimento Orientado por Testes (Test-Driven Development ou TDD)

- é uma abordagem de desenvolvimento de software que inverte o processo tradicional de codificação. Em vez de escrever o código primeiro e depois testá-lo, o TDD propõe que os testes sejam escritos antes do código de implementação.
- O Ciclo do TDD (Red-Green-Refactor):
 - Red (Vermelho): Escreva um teste que falhe para uma funcionalidade ainda não implementada.
 - Green (Verde): Escreva o código mínimo necessário para fazer o teste passar.
 - Refactor (Refatorar): Melhore o código, mantendo os testes passando.

Benefícios



Código mais confiável: Garantia de que o código funciona conforme o esperado.



Design melhor: O código tende a ser mais modular e com baixo acoplamento.



Documentação viva: Os testes servem como documentação do comportamento esperado.



Feedback rápido: Problemas são identificados imediatamente.



Refatoração segura: Mudanças podem ser feitas com confiança.



Menos bugs: Problemas são identificados e corrigidos mais cedo.



Desenvolvimento mais focado: Clareza sobre o que precisa ser implementado.

Desafios do TDD:



Tipos de Testes

- Testes Unitários:
 - Testam unidades individuais de código (funções, métodos, classes).
 - São rápidos, isolados e não dependem de recursos externos.
 - Geralmente usam mocks para isolar a unidade testada.
 - Exemplo: testar uma função de cálculo de imposto.

Testes de Integração:

Testam a interação entre diferentes partes do sistema.



Verificam se os componentes funcionam bem juntos.

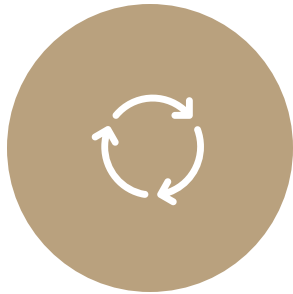


Podem envolver bancos de dados, APIs ou outros serviços.

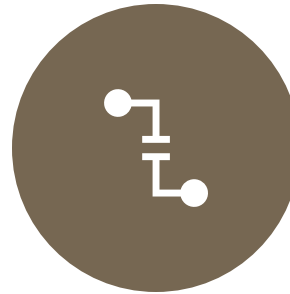


Exemplo: testar se um serviço consegue salvar dados no banco.

Testes End-to-End (E2E)



Testam o fluxo completo da aplicação, simulando o comportamento do usuário.



São mais lentos e mais propensos a falhas.



Geralmente envolvem navegadores reais ou simulados.



Exemplo: testar o processo de login até a compra em um e-commerce.

Testes de Aceitação

Verificam se o sistema atende aos requisitos do negócio.

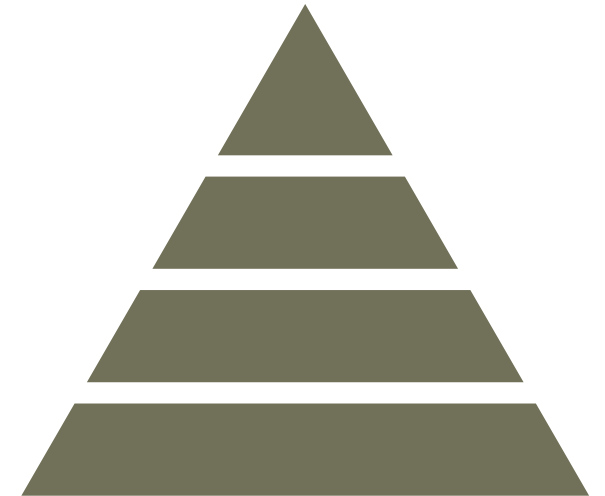
Geralmente escritos em linguagem mais próxima do usuário.

Podem ser automatizados ou manuais.

Exemplo: verificar se o sistema permite que um usuário complete uma compra.

Pirâmide de Testes

- A pirâmide de testes sugere uma proporção entre os diferentes tipos de testes
 - Base (maior quantidade): Testes Unitários
 - Meio: Testes de Integração
 - Topo (menor quantidade): Testes E2E/UI



Ferramentas de Teste para JavaScript/TypeScript: Jest



Desenvolvido pelo Facebook



Zero configuração para começar



Inclui runner de testes, assertions, mocks e cobertura de código



Snapshots para testar UI



Execução paralela de testes

Bibliotecas de Assertions

Assert (Node.js):

- Biblioteca nativa do Node.js
- Estilo mais simples
- Exemplo:
`assert.strictEqual(result, 5)`

Jest Expect:

- Integrado ao Jest
- Sintaxe expressiva
- Exemplo: `expect(result).toBe(5)`

Bibliotecas de Mocks (Jest Mocks)



Sistema de mocking
integrado ao Jest



Mocks automáticos
de módulos



API para funções
mock e timers

Ferramentas de Cobertura de Código

Istanbul:

- Ferramenta de cobertura de código para JavaScript
- Integração com a maioria dos frameworks de teste
- Relatórios detalhados (statements, branches, functions, lines)

Jest Coverage:

- Integrado ao Jest
- Baseado no Istanbul
- Configurável via package.json ou jest.config.js

Boas Práticas de TDD

Mantenha os testes simples: Um teste deve verificar uma única coisa.



Testes independentes: Cada teste deve poder ser executado isoladamente.



Testes determinísticos: O mesmo teste deve sempre produzir o mesmo resultado.



Testes rápidos: Testes lentos desencorajam a execução frequente.



Testes legíveis: O propósito do teste deve ser claro.

Estrutura de Testes

Arrange-Act-Assert (AAA):

- Arrange: Configurar as pré-condições e entradas
- Act: Executar o código a ser testado
- Assert: Verificar os resultados esperados

Given-When-Then (GWT):

- Given: Contexto inicial (pré-condições)
- When: Evento ou ação que ocorre
- Then: Resultado esperado

Nomeação de Testes

Bons nomes
de testes
descrevem:

- O que está sendo testado
- Em que circunstâncias
- Qual o resultado esperado

Exemplos:

- `should_returnTrue_when_inputIsValid()`
- `calculateTotal_withDiscountApplied_returnsDiscounted Price()`
- `getUserById_nonExistentUser_throwsNotFoundException()`

Evitando Testes Frágeis



Não teste implementações internas: Teste comportamentos, não detalhes de implementação.



Evite acoplamento excessivo: Testes muito acoplados quebram com pequenas mudanças.



Use fixtures e factories: Para criar dados de teste consistentes.



Isole dependências externas: Use mocks ou APIs de teste dedicadas.



Evite testes não determinísticos: Cuidado com datas, números aleatórios, etc.

Refatoração de Testes

Extraia código repetitivo: Use helpers, fixtures e setup/teardown.



Mantenha a clareza: A legibilidade é mais importante que a DRY (Don't Repeat Yourself).



Refatore incrementalmente: Mude um teste de cada vez.



Mantenha a cobertura: Não remova casos de teste durante a refatoração.

```
// Grupo de testes
describe('Component/Function Name', () => {
  // Setup antes de todos os testes
  beforeAll(() => {
    // Configuração global
  });
  // Setup antes de cada teste
  beforeEach(() => {
    // Configuração para cada teste
  });
  // Cleanup após cada teste
  afterEach(() => {
    // Limpeza após cada teste
  });
  // Cleanup após todos os testes
  afterAll(() => {
    // Limpeza global
  });
  // Teste individual
  it('should do something specific', () => {
    // Arrange
    const input = 'test';
    // Act
    const result = someFunction(input);
    // Assert
    expect(result).toBe('expected output');
  });
});
```

```
// Teste assíncrono
it('should handle async operations', async () => {
  // Arrange
  const input = 'test';

  // Act
  const result = await someAsyncFunction(input);

  // Assert
  expect(result).toEqual({ success: true });
});

// Teste com mock
it('should call dependencies correctly', () => {
  // Arrange
  const mockDependency = jest.fn().mockReturnValue('mocked result');

  // Act
  const result = functionWithDependency(mockDependency);

  // Assert
  expect(mockDependency).toHaveBeenCalledTimes(1);
  expect(mockDependency).toHaveBeenCalledWith('expected argument');
  expect(result).toBe('mocked result');
});
});
```

// Igualdade

`expect(value).toBe(otherValue);` *// Igualdade estrita (===)*

`expect(value).toEqual(otherValue);` *// Igualdade profunda (deep equality)*

`expect(value).not.toBe(otherValue);` *// Negação*

// Verdade/Falsidade

`expect(value).toBeTruthy();`

`expect(value).toBeFalsy();`

`expect(value).toBeNull();`

`expect(value).toBeUndefined();`

`expect(value).toBeDefined();`

// Números

`expect(value).toBeGreaterThan(3);`

`expect(value).toBeGreaterThanOrEqual(3);`

`expect(value).toBeLessThan(3);`

`expect(value).toBeLessThanOrEqual(3);`

`expect(value).toBeCloseTo(0.3);` *// Para números de ponto flutuante*

// Strings

`expect(value).toMatch(/pattern/);`

`expect(value).toContain('substring');`

// Arrays

`expect(array).toContain(item);`

`expect(array).toHaveLength(3);`

Matchers Comuns do Jest:

// Objetos

`expect(object).toHaveProperty('property');`

`expect(object).toHaveProperty('nested.property', 'value');`

// Exceções

`expect(() => { throw new Error('test'); }).toThrow();`

`expect(() => { throw new Error('test'); }).toThrow('test');`

`expect(() => { throw new Error('test'); }).toThrow(/test/);`

// Funções Mock

`expect(mockFn).toHaveBeenCalled();`

`expect(mockFn).toHaveBeenCalledTimes(2);`

`expect(mockFn).toHaveBeenCalledWith(arg1, arg2);`

`expect(mockFn).toHaveBeenLastCalledWith(arg1, arg2);`

React Testing Library

React Testing Library (RTL) é uma biblioteca para testar componentes React de uma maneira que se assemelha à forma como os usuários interagem com a aplicação.

Ela encoraja boas práticas de teste, focando no comportamento em vez de detalhes de implementação.

Princípios da Testing Library

Teste o comportamento, não a implementação: Foque no que o usuário vê e faz.

Encontre elementos como um usuário faria: Por texto, rótulos, roles, etc.

Evite testar detalhes de implementação: Não teste props, estado ou métodos internos.

Prefira queries acessíveis: Use `getByRole`, `getByText`, etc. em vez de `getById`.

Queries Principais

// Queries por texto

`screen.getByText('Hello, World');` *// Encontra elemento com texto exato*

`screen.getByText(/hello/i);` *// Suporta regex*

// Queries por role (acessibilidade)

`screen.getByRole('button');` *// Encontra um botão*

`screen.getByRole('button',{ name: 'Submit' });` *// Botão com nome específico*

// Queries por label

`screen.getByLabelText('Username');` *// Encontra input pelo label*

// Queries por placeholder

`screen.getByPlaceholderText('Enter username');`

// Queries por test ID (último recurso)

`screen.getByTestId('submit-button');`

// Variantes das queries:

// getBy - Encontra um elemento ou lança erro*

// queryBy - Encontra um elemento ou retorna null (bom para verificar ausência)*

// findBy - Versão assíncrona de getBy* (espera elemento aparecer)*

// getAllBy, queryAllBy*, findAllBy* - Versões que retornam arrays*

Simulando Interações do Usuário

```
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import Counter from './Counter';

test('increments counter when button is clicked', async () => {
  // Configurar userEvent
  const user = userEvent.setup();

  // Renderizar componente
  render(<Counter initialCount={0} />);

  // Verificar valor inicial
  expect(screen.getByText('Count: 0')).toBeInTheDocument();

  // Encontrar botão e clicar
  const button = screen.getByRole('button', { name: 'Increment' });
  await user.click(button);

  // Verificar valor após clique
  expect(screen.getByText('Count: 1')).toBeInTheDocument();
});
```


Testando Formulários

```
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import LoginForm from './LoginForm';

test('submits form with user data', async () => {
  // Mock da função de submit
  const handleSubmit = jest.fn();
  const user = userEvent.setup();

  // Renderizar componente
  render(<LoginForm onSubmit={handleSubmit} />);

  // Preencher campos
  await user.type(screen.getByLabelText('Email'), 'test@example.com');
  await user.type(screen.getByLabelText('Password'), 'password123');

  // Submeter formulário
  await user.click(screen.getByRole('button', { name: 'Login' }));

  // Verificar se a função foi chamada com os dados corretos
  expect(handleSubmit).toHaveBeenCalledWith({
    email: 'test@example.com',
    password: 'password123'
  });
});
```

Testando Componentes Assíncronos

```
import { render, screen } from '@testing-library/react';  
import UserProfile from './UserProfile';
```

```
// Mock do módulo de API
```

```
jest.mock('../api/userApi', () => ({  
  fetchUserProfile: jest.fn(() => Promise.resolve({  
    id: 1,  
    name: 'John Doe',  
    email: 'john@example.com'  
  })),  
}));
```

```
test('loads and displays user profile', async () => {
```

```
// Renderizar componente
```

```
  render(<UserProfile userId={1} />);
```

```
// Verificar estado de carregamento
```

```
  expect(screen.getByText('Loading...')).toBeInTheDocument();
```

```
// Esperar dados carregarem
```

```
  expect(await screen.findByText('John Doe')).toBeInTheDocument();
```

```
  expect(screen.getByText('john@example.com')).toBeInTheDocument();
```

```
// O elemento de loading não deve mais estar presente
```

```
  expect(screen.queryByText('Loading...')).not.toBeInTheDocument();
```

```
});
```

```
// Button.tsx
import React from 'react';
interface ButtonProps {
  label: string;
  onClick: () => void;
  disabled?: boolean;
}
const Button: React.FC<ButtonProps> = ({ label, onClick,
disabled = false }) => {
  return (
    <button
      onClick={onClick}
      disabled={disabled}
      className={btn ${disabled ? 'btn-disabled' : 'btn-primary'}}
    >
      {label}
    </button>
  );
};
export default Button;
```

```
// Button.test.tsx
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import Button from './Button';
describe('Button', () => {
  test('renders with correct label', () => {
    render(<Button label="Click me" onClick={jest.fn()} />);
    expect(screen.getByRole('button', { name: 'Click me' })).toBeInTheDocument();
  });
});
```

```
test('calls onClick when clicked', async () => {
  const handleClick = jest.fn();
  const user = userEvent.setup();
  render(<Button label="Click me" onClick={handleClick} />);
  await user.click(screen.getByRole('button'));
  expect(handleClick).toHaveBeenCalledTimes(1);
});
test('is disabled when disabled prop is true', async () => {
  const handleClick = jest.fn();
  const user = userEvent.setup();
  render(<Button label="Click me" onClick={handleClick} disabled={true} />);
  const button = screen.getByRole('button');
  expect(button).toBeDisabled();
  await user.click(button);
  expect(handleClick).not.toHaveBeenCalled();
});
test('has correct CSS classes', () => {
  const { rerender } = render(<Button label="Click me" onClick={jest.fn()} />);
  expect(screen.getByRole('button')).toHaveClass('btn');
  expect(screen.getByRole('button')).toHaveClass('btn-primary');
  expect(screen.getByRole('button')).not.toHaveClass('btn-disabled');
  rerender(<Button label="Click me" onClick={jest.fn()} disabled={true} />);
  expect(screen.getByRole('button')).toHaveClass('btn');
  expect(screen.getByRole('button')).toHaveClass('btn-disabled');
  expect(screen.getByRole('button')).not.toHaveClass('btn-primary');
});
});
```

Testando Componentes React

Cobertura de Testes

A cobertura de testes
mede quanto do
código está sendo
testado.

É uma métrica
importante para garantir
que partes críticas do
código estejam
cobertas por testes.

Interpretando Relatórios de Cobertura

O relatório de cobertura do Jest inclui várias métricas:

- **Statements:** Percentual de declarações executadas
- **Branches:** Percentual de ramificações condicionais executadas
- **Functions:** Percentual de funções chamadas
- **Lines:** Percentual de linhas de código executadas

O relatório também mostra quais linhas específicas não foram cobertas, ajudando a identificar áreas que precisam de mais testes.

Estratégias para Melhorar a Cobertura

Identificar áreas críticas: Priorize a cobertura de código que:

- Contém lógica de negócio complexa
- É propenso a erros
- Muda frequentemente
- Tem impacto direto nos usuários

Testar caminhos alternativos: Não teste apenas o "caminho feliz", mas também:

- Condições de erro
- Valores limite
- Casos extremos
- Entradas inválidas

Usar TDD: Escrever testes antes do código garante cobertura desde o início.

Revisar testes regularmente: À medida que o código evolui, os testes devem ser atualizados.

Integrar na CI/CD: Falhar o build se a cobertura cair abaixo de um limite aceitável.

Limitações da Cobertura de Código

Alta cobertura não garante ausência de bugs

Testes de baixa qualidade podem ter alta cobertura

Algumas partes do código são difíceis de testar (UI, integrações externas)

Focar demais em métricas pode levar a testes artificiais